

電子工作連携 詳細実装設計書

fighter-game x Raspberry Pi | 2026-09-12

1. 概要

fighter-game(スマブラ風2D対戦ゲーム)にGPIOボタン・NeoPixel LED・MPU-6050加速度センサーを接続し、物理筐体体験を追加する。

2. システムアーキテクチャ

レイヤー	コンポーネント	役割
フロントエンド	index.html / src/*.js	ゲーム描画・WebSocket送信(イベント通知)
ブリッジ層	hardware_server.py	WebSocketサーバー(ws://localhost:8765)・GPIO/I2C制御
入力	タクトスイッチ x10	GPIO入力 -> xdotool キーイベント注入
出力	NeoPixel LEDテープ	rpi_ws281x -> ゲームイベントで発光
入力	MPU-6050 x2	I2C(smbus2) -> 傾き/振り -> 操作キー変換

3. ハードウェア詳細設計

3.1 タクトスイッチ GPIO割り当て

役割	GPIO	物理Pin	接続先
1P 左移動	17	11	GND(9)
1P 右移動	25	22	GND(14)
1P ジャンプ	27	13	GND(14)
1P しやがみ	22	15	GND(14)
1P 弱攻撃	23	16	GND(20)
1P 横強攻撃	24	18	GND(20)
2P 左移動	5	29	GND(25)
2P 右移動	6	31	GND(30)
2P ジャンプ	13	33	GND(34)
2P しやがみ	19	35	GND(34)
2P 弱攻撃	26	37	GND(39)
2P 横強攻撃	21	40	GND(39)

3.2 NeoPixel

信号	ラズパイ側	部品	備考
DATA	GPIO18(Pin12)	470Ω直列抵抗経由	PWM対応ピン必須
VCC	5V(Pin2 or 4)	1000μFコンデンサ並列	大電流対策
GND	GND(Pin6)	共通GND	ラズパイと共通

3.3 MPU-6050 I2C

信号	ラズパイ側	デバイス側	備考
VCC	3.3V(Pin1)	VCC	3.3V動作

信号	ラズパイ側	デバイス側	備考
GND	GND(Pin6)	GND	共通GND
SDA	GPIO2/SDA(Pin3)	SDA	I2Cデータ
SCL	GPIO3/SCL(Pin5)	SCL	I2Cクロック
1P AD0	-	GND接続	I2Cアドレス 0x68
2P AD0	-	3.3V接続	I2Cアドレス 0x69

4. ソフトウェア実装設計

4.1 ファイル構成

```
fighter-game/
+-- src/hardware_bridge.js    ■■■: WebSocket■■■■■■■
+-- hardware/
    +-- hardware_server.py    ■■■: ■■■■■■■■(WS+GPIO+LED+IMU)
    +-- gpio_controller.py    ■■■: ■■■■■■->xdotool
    +-- led_controller.py     ■■■: NeoPixel■■■
    +-- imu_controller.py     ■■■: MPU-6050->xdotool
    +-- requirements.txt      ■■■: rpi-ws281x, smbus2, websockets
```

4.2 WebSocket イベントスキーマ

方向	イベントtype	主要フィールド	用途
JS->Server	hit	player, damage	ヒット時LED発光
JS->Server	ko	player, stocks_left	KO時LED演出
JS->Server	game_start	-	開始演出
JS->Server	game_end	winner	終了演出
JS->Server	damage_update	p1, p2	常時LED更新

4.3 src/main.js 変更箇所

場所	追加コード
import部	import { HardwareBridge } from './hardware_bridge.js';
初期化	const hw = new HardwareBridge();
ヒット判定	hw.send('hit', {player, damage});
KO判定	hw.send('ko', {player, stocks_left});
ゲームループ末尾	hw.send('damage_update', {p1: f1.damage, p2: f2.damage});

4.4 GPIO入力変換ロジック (gpio_controller.py)

```
KEY_MAP = {17:"a", 25:"d", 27:"w", 22:"s", 23:"f", 24:"g",
           5:"Left", 6:"Right", 13:"Up", 19:"Down", 26:"shift", 21:"ctrl"}

def poll(self): # 10ms■■■■■■■
    for pin, key in KEY_MAP.items():
        val = GPIO.input(pin)
        if val != self._state[pin]:
            action = "keydown" if val==GPIO.LOW else "keyup"
            subprocess.Popen(["xdotool", action, "--clearmodifiers", key])
            self._state[pin] = val
```

4.5 LEDアニメーション仕様

イベント	アニメーション	持続
hit(P1受け)	P1側30LEDが赤点滅	200ms
hit(P2受け)	P2側30LEDが青点滅	200ms
ko	全体虹色スクロール	1000ms
game_start	全体グリーン->フェードアウト	800ms
damage_update	残ストック数を先頭/末尾LEDで常時表示	常時
アイドル	中央20LEDが白dimで呼吸	常時

4.6 IMU入力変換ロジック (imu_controller.py)

検出	閾値	操作	備考
X軸傾き	±15度	左右移動	atan2(ax,az)で計算
加速度スパイク	delta>2.0g	攻撃	前フレームとの差分

5. 依存ライブラリ

ライブラリ	用途	インストール
RPi.GPIO	GPIOボタン入力	apt: python3-rpi.gpio
rpi-ws281x	NeoPixel LED制御	pip install rpi-ws281x
smbus2	MPU-6050 I2C読み取り	pip install smbus2
websockets	WebSocketサーバー	pip install websockets
xdotool	キーイベント注入	apt install xdotool

6. 起動手順

手順	コマンド	備考
①ライブラリインストール	<code>pip install -r hardware/requirements.txt</code>	初回のみ
②I2C確認	<code>sudo i2cdetect -y 1</code>	0x68/0x69表示確認
③ゲームサーバー起動	<code>python3 -m http.server 8000</code>	バックグラウンド
④HWサーバー起動	<code>sudo python3 hardware/hardware_server.py</code>	NeoPixelはsudo必須
⑤ブラウザ	<code>chromium-browser http://localhost:8000</code>	全画面F11推奨

7. 注意事項

- ・ NeoPixelのVCCは5V必須。60LED以上の場合は外部5V電源(2A以上)を推奨。
- ・ 3.3V GPIO信号で動作しないNeoPixelには74HCT125でレベルシフトが必要。
- ・ MPU-6050のVCCは3.3VとしI2Cバス電圧に合わせる。
- ・ hardware_server.pyはsudo実行が必要(NeoPixelのDMA制御のため)。
- ・ xdotoolはアクティブウィンドウへ送信するため、Chromiumが前面にある必要がある。